GitHub Copilot Hackathon

🎯 Objective

Experience how GitHub Copilot can accelerate modern software development by building a simple, end-to-end application and using Playwright to validate it within a few hours.

Participants will use Copilot to:

- Rapidly scaffold application components (frontend + API)
- Implement core functionality using natural language prompts
- Apply basic business logic or validation to real-world scenarios
- Generate, refine, and troubleshoot meaningful Playwright end-to-end tests

Theme: Document Validation

Build an app that validates documents such as HR forms, invoices, certificates, or ID documents.

> ***The goal is not to build a production-ready system, but to demonstrate how AI-assisted development can significantly reduce effort, improve productivity, and enable faster innovation.***

👥 Participants

- Work individually, with optional pairing if preferred
- Optional: work in a **GitHub repository** to collect artifacts after the event

📋 Pre-Requisites (Send 3+ Days Before)

Participants must have the following ready **before** the hackathon day:

- [ ] IDE that supports GitHub Coplot [VS Code recommended] installed
- [ ] [GitHub Copilot extension](https://marketplace.visualstudio.com/items?itemName=GitHub.copilot) installed and licensed
- [ ] Git installed
- [ ] Runtime of choice installed (Node.js 20+, Python 3.11+, or .NET 8+)
- [ ] Familiarity with at least one web framework

🗓️ Agenda

Phase	Duration
Kickoff & Setup	30 min
Build Phase 1 — Core App	1.5 hours
Build Phase 2 — Playwright Foundations	1.5 hours
Build Phase 3 — Playwright Deepening	1.5 hours

Demos & Wrap-Up	1 hour

Kickoff & Setup (30 min)

- Welcome and session goals
- Overview of the challenge and example use cases
- Verify environments are working

Build Phase 1 — Core App (1.5 hours)

Goal: **Achieve a working end-to-end flow.**

- Create a simple frontend (upload form or input UI)
- Build an API endpoint to receive and process data
- Implement basic data extraction or parsing logic
- Connect frontend → API → response displayed to user

> *💡 **Tip:** Use a starter template (see below) to skip boilerplate and jump straight to the interesting parts.*

Build Phase 2 — Playwright Foundations (1.5 hours)

Goal: **Create the first meaningful Playwright coverage for the app.**

- Install and configure Playwright
- Create the first end-to-end spec with Copilot
- Automate the happy path for the chosen document-validation scenario
- Run tests locally and interpret failures with Copilot

Build Phase 3 — Playwright Deepening (1.5 hours)

Goal: **Deepen coverage and improve test quality.**

- Add at least one negative-path validation test
- Add one richer scenario such as file upload, multi-document flow, retry/error handling, or mocked backend behavior
- Improve selectors and assertions using accessible locators such as `getByRole` and `getByLabel`
- Use trace or screenshot output to debug a failed test with Copilot

Demos & Wrap-Up (1 hour)

- Team presentations (5–8 minutes each)
- Share what worked, what surprised you, and favorite Copilot moments
- Feedback survey

Example Use Cases

Pick **one** to keep scope manageable:

Use Case	What It Validates
Invoice Checker	Required fields present (vendor, amount, date, PO number), amounts are positive, date is not in the future
HR Form Validator	Employee name, ID, department filled in; signature present; dates are logical
ID Document Verifier	Expiry date check, required fields present, format consistency
Certificate Validator	Issuer information present, valid date range, certificate number format
Expense Report Auditor	Line items sum to total, receipts attached, within policy limits

Recommended Starter Stacks

Teams are free to choose their own, but here are three fast-start paths:

Stack	Frontend	Backend	Good For
JavaScript	React or plain HTML	Node.js + Express	JS/TS-comfortable teams
Python	HTML + Jinja or React	Flask / FastAPI	Data/ML-leaning teams
Java	Thymeleaf or React	Spring Boot	Java teams
.NET	Blazor or Razor Pages	ASP.NET Core Web API	.NET teams

Scope Guidance

To ensure success within the session:

- **Use Copilot throughout** — the point is the experience, not just the output
- **Focus on one document type / use case** — don't try to do everything
- **Prioritize a working solution over complexity** — a simple app that works beats a complex one that doesn't

What Success Looks Like

- ✅ A working application (frontend + API) running locally
- ✅ Three meaningful Playwright test scenarios: happy path, negative path, and one deeper scenario
- ✅ Clear demonstration of Copilot-assisted development during the presentation
- ✅ Evidence that the team used Copilot to generate, refine, and debug Playwright tests
- ✅ A practical, business-relevant validation scenario
- ✅ A team that learned something new about AI-assisted development